Advanced Cognitive Modelling

Assignment 2

This report describes a cognitive model in the matching pennies game. We build a single-agent learning model in which the agent forms and updates beliefs about the opponent agent's tendency to choose a particular "hand" or option. This belief is modelled to be updated trial by trial using an exponential learning rule. The model is implemented and fitted using Stan, which allows us to estimate the central cognitive parameters: Learning rate, choice sensitivity, and bias. To evaluate the quality of the model, we do prior and posterior predictive checks, examine prior-posterior updates, and finally we do parameter recovery. These steps together, help us assess whether the model provides a reliable account of cognition and behaviour in the matching pennies task.

Model Description

The model is built so the agent learns about the opponent's behaviour over time by forming and updating an internal belief about how likely the opponent agent is to pick a particular option (coded as 1). This belief is being stored as a memory variable that indicates the agent's current expectation. After each trial, the agent observes the opponent's choice and then updates the belief. The agent then uses this belief to make its own decisions. The model is made so choices are not fully deterministic. Instead, the agent makes probabilistic decisions, which means that even with a strong belief, there is still some variability in its behaviour. This allows us to observe both learning and noise in the agent's decision making.

On each trial, the agent begins with its current belief about the opponent's choice. Based on this belief, the model computes the probability of choosing 1. This depends on two factors: The current belief and a general bias towards one of the options.

The agent's choice is then sampled from this probability, meaning that choices can vary despite similar conditions. After the agent has observed the opponent's actual choice, it updates its belief. The update depends on the difference between what the agent expected and what was actually observed. This also leads to larger updates when the prediction error is large. What we observe is that this process allows the agent to observe the opponent's choices and track them over trials.

Parameters

Learning rate (α)	Describes how quickly the agent updates its belief based on new observations.
Choice sensitivity (β)	Indicates how strongly the internal belief affects the agent's decisions.
Bias	Tendency to pick one option over another
Initial memory (m0)	The agent's starting belief before any observations are made.

Stan model description

See Appendix A for the full Stan code or go to Github:
https://github.com/AmalieOvergaard/Adv-Cog-Modelling-Portfolio-2

Discussion of model quality

Prior predictive check

We use the prior predictive check to infer whether the model and the priors can generate reasonable data, before seeing the actual observed data. The opponent agent is modelled to choose option 1 with a probability of 0.6. We expect our agent to learn this pattern over trials and thereby achieve to match the opponent with a probability above 0.5. Looking at figure 1, most trajectories fall around or above 0.5. A success rate of 0.5 corresponds to chance-level performance, where agents do not match the opponent better than random guessing would. This means that values above 0.5 indicate that the is successfully learning and exploiting the opponent's bias, while values below 0.5 reflect poor behaviour. In our prior predictive simulations, some fall below, indicating that the priors allow both effective and less effective behaviour.

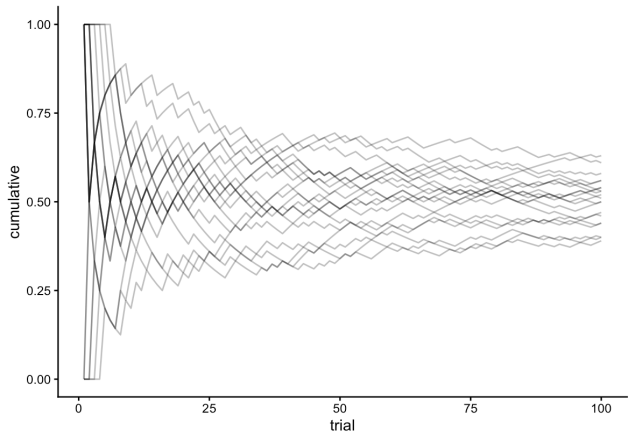

Figure 1: Prior predictive check

To test whether the poorer performance in some of the simulations was driven by wide priors, we repeated the prior predictive check with slightly tighter priors on alpha and beta. The results in figure 2 show a similar pattern overall, indicating that the priors are not solely causing the variability in behaviour, but is also inherent in the model.

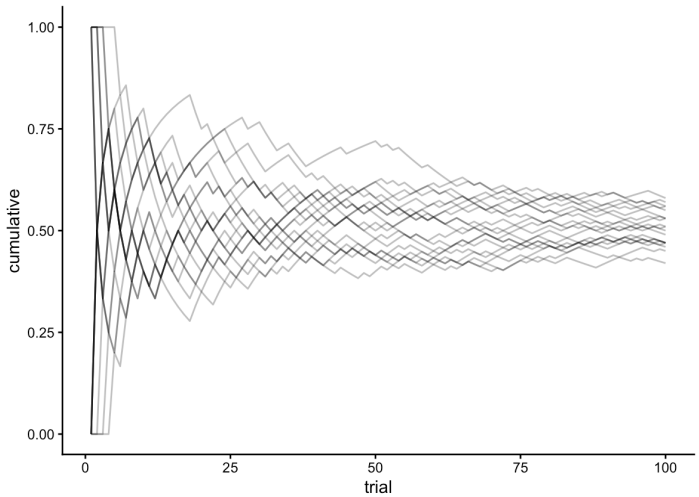

Figure 2: Prior predictive check (tighter priors on beta and alpha)

Posterior predictive check

We use the posterior predictive check to test whether the model captures the structure from the observed data. As illustrated in figure 3, the observed data (black line) lies within the range of the simulated data (grey lines), suggesting that the model captures the cumulative success pattern over trials. From this result we can infer that the model appears to reproduce key tendencies, although it cannot fully explain all aspects of the observed data.

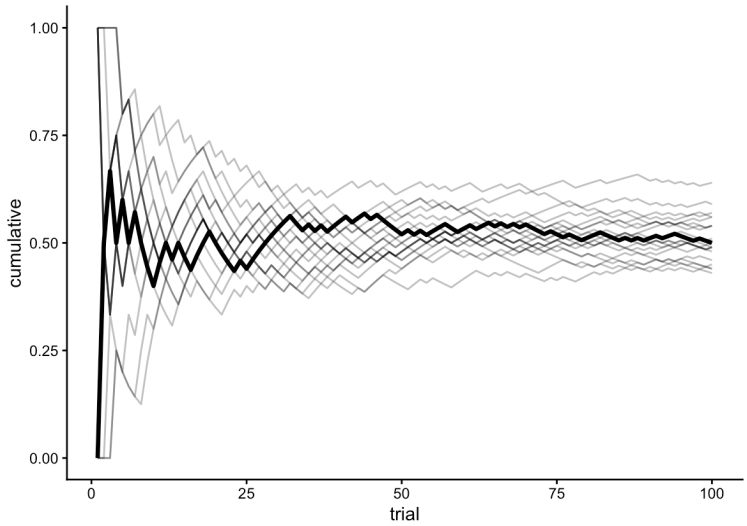

Figure 3: Posterior predictive check

Prior vs posterior

The comparison of the priors and posteriors in figure 4, shows a narrower and posterior and a shift towards lower values, for the alpha parameter. This means that the model is more certain about the alpha and that the data reduced the uncertainty. The shifted posterior also tells us that the model believes that lower alpha values are more likely than what we assumed in our priors. Connecting that to behaviour, a lower alpha means slower learning. In contrast, the posterior for beta looks similar to the prior, which tells us that the data did not strongly make us update our belief about the parameter.

For the bias parameter, the posterior is much more concentrated around 0 than the prior expects. This indicates that the parameter is well identified and that the data suggests almost no side bias in the agent's choices. The m0 parameter shows a slight shift towards higher values relative to the prior, suggesting that higher initial memory values are most consistent than the priors assume. In terms of behaviour, this means that the agent initially expects the opponent to choose option 1 more than chance.

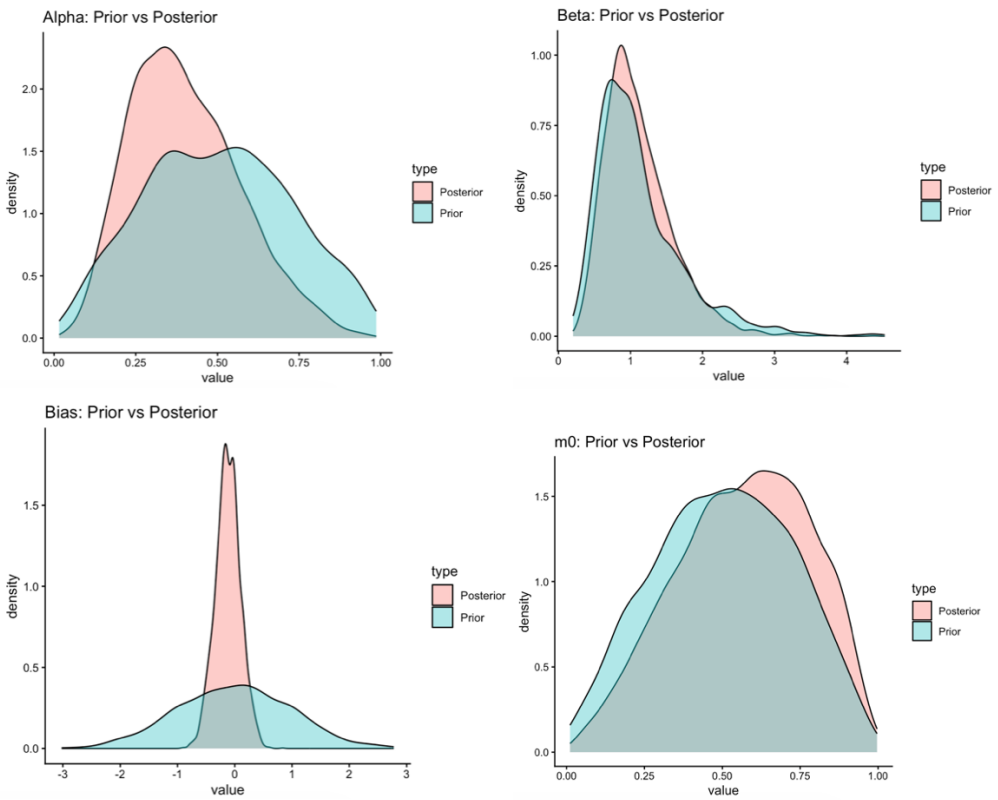

Figure 4: Prior vs posterior: alpha, beta, bias, m0

Parameter recovery

We perform recovery to see whether the model can recover the parameters that is not directly observable. Testing these parameters is an essential part of evaluating a model, as it helps us uncover whether the model is correctly inferring the cognitive processes that we are trying to model.

To assess the recovery, we start by a range of known values for the parameter that we want to test. We do this to test different values across the parameter space rather than just one value. To isolate the recovery of a chosen parameter, we fix the remaining to known values, which mean that for alpha recovery, beta, bias, and m0 were held constant. This makes the process much easier to interpret, as differences in the estimates can then be attributed mainly to the parameter being varied.

For each of the set true values, we then generate data with the same model as we will later fit in Stan, meaning that we simulate an opponent and then we simulate an agent and its choices trial by trial. This process is then repeated several times (n_reps = 20) which allows us to assess the variation in the data. We then prepare the simulated choices to work as Stan input and fit the same model. We can then extract

the posterior mean for the parameter that we chose to recover. Finally, we plot the recovered estimates against the true values. If the recovery went well, the estimated values lie close to the diagonal line.

Results

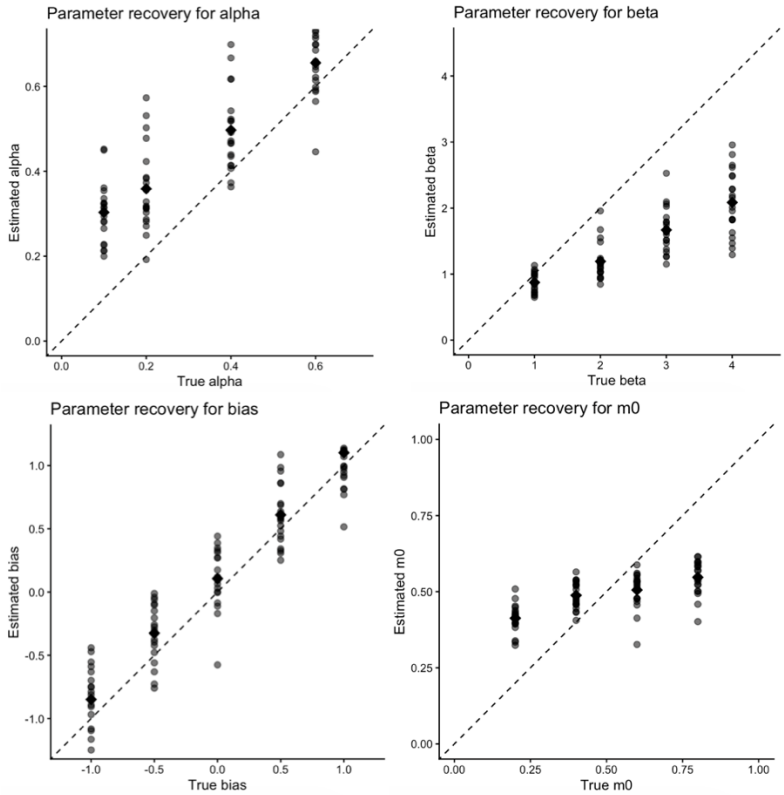

Figure 5: Parameter recovery for alpha, beta, bias, m0

Alpha

Looking at the alpha recovery plot, we can interpret that the estimated values generally follow the diagonal line, indicating that the parameter is fairly well recovered. However, the estimates tend to lie above the line which suggest an overestimation of alpha.

Beta

From the beta recovery plot, we can infer that the values did not recover as well as alpha. Estimated values appear to be systematically too low as true beta increases. This means that the model underestimates higher beta values by compressing everything towards lower values. What the parameter recovery of beta tells us is that the parameter does not scale well as the model shows limited abilities to recover larger values.

Bias

The parameter recovery for bias shows points lying diagonal across the full range. There is also no systematic bias, which all around indicates a very good recovery. This suggests that the bias parameter was well identified by the model. It can be argued that bias is easier to recover than beta, as the parameter is constant and not dependent on memory as beta is.

m0

From the m0 recovery plot, we can infer that high values are underestimated as for beta. The parameter is partially recovered, but there seems to be a bias towards intermediate values. An explanation could be that m0 only matters in the beginning of the trials and after that, memory is quickly updated by alpha. This means that m0 influence fades, which can explain the weaker recovery.

Overall, the parameter recovery results suggest that parameters with a direct influence on choice behaviour, such as bias, are well recovered. In contrast, parameters that depend on the memory, such as beta and m0, show underestimation at higher values and are more difficult to recover accurately. Alpha shows relatively better recovery, although some bias toward intermediate values is still occurring. This pattern tells us that parameters related to memory processes are harder to identify than those with a more direct effect on behaviour.

Number of trials needed

To get an indication of how many trials we would need to recover parameters accurately, we varied the number of trials ranging from 20 to 200 and assessed the parameter recovery for each condition. The results show that with a low number of trials (20-50), we see larger variation in the estimated parameters. As the number of trials increases, we observe the values become more stable and closer to the true values. This improvement is the most evident for bias and m0, where estimated values seem to cluster around true values as the number of trials increases.

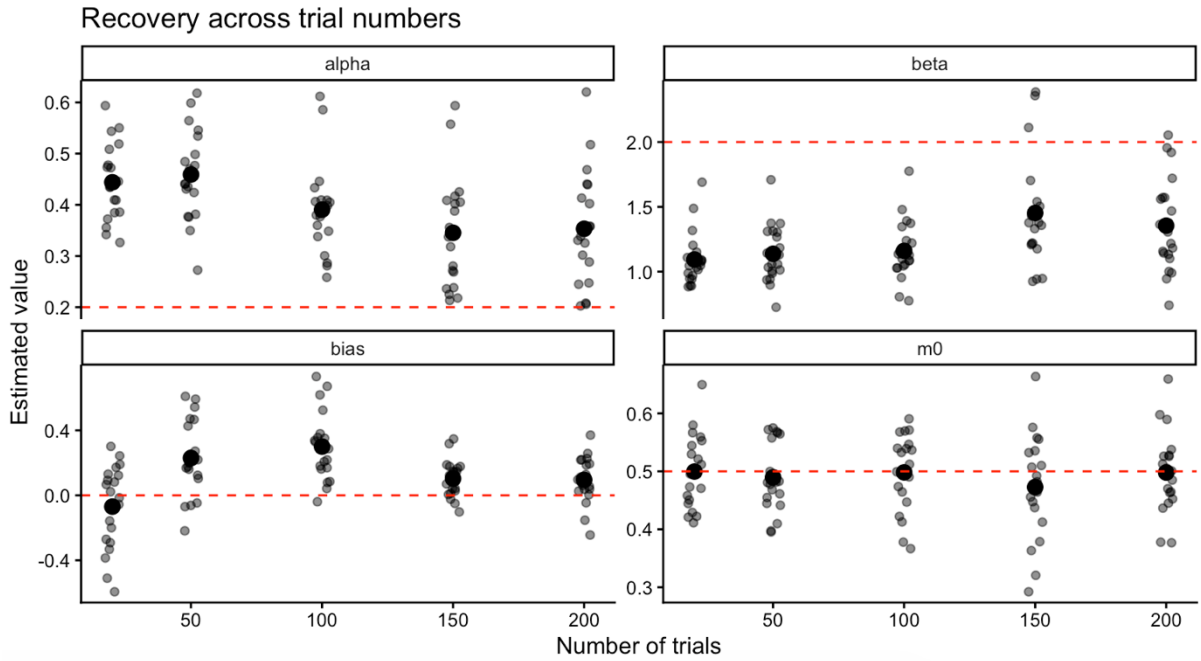

Figure 6: Number of trials needed

The role of priors

The role of priors in the recovery results appears to be that they contribute to a shrinkage of parameter estimates toward central values. This is particularly evident for parameters such as alpha, beta, and m0, where extreme values tend to be underestimated and estimates cluster around middle values. This effect is more clear for parameters that are harder to identify from the data, especially those related to the internal memory of the model. In contrast, parameters with a more direct influence on behaviour, such as bias, are less affected by the priors. Together, this tells us that priors play an important role in stabilizing estimates but can also introduce bias when the data are limited or less informative.

MCMC Diagnostics

To assess the reliability of our parameter estimation, we examined the MCMC diagnostics. For all parameters, the $\hat{R}$ values are equal to 1 which indicate a robust convergence.

Appendix

A: Full commented Stan code

```
data {
  int<lower=1> T;
  // Number of trials in the task

  array[T] int<lower=0, upper=1> Self;
  // The agent's observed choices on each trial (0 or 1)

  array[T] int<lower=0, upper=1> Other;
  // The opponent's observed choices on each trial (0 or 1)
}

parameters {
  real<lower=0, upper=1> alpha;
  // Learning rate: controls how strongly new opponent choices update memory

  real<lower=0> beta;
  // Choice sensitivity: controls how strongly memory influences choice probability

  real bias;
  // General side bias: captures a tendency to prefer one response side

  real<lower=0, upper=1> m0;
  // Initial memory state before the first trial
}

transformed parameters {
  vector[T] eta;
  // Linear predictor (log-odds of choosing 1) for each trial

  vector[T] p;
  // Choice probability of choosing 1 on each trial

  vector[T] memory_before;
  // Memory state before observing the opponent on each trial

  vector[T] memory_after;
  // Memory state after updating from the opponent's choice

  real m_now;
  // Temporary variable storing the current memory state as the model moves through trials
```

```
  m_now = m0;
  // Initialize memory at the starting value m0

  for (t in 1:T) {
    memory_before[t] = m_now;
    // Store the current memory before making the update on this trial

    eta[t] = bias + beta * (2 * (m_now - 0.5));
    // Compute the log-odds of choosing 1
    // Memory is centered around 0.5, so values above 0.5 favor choosing 1
    // and values below 0.5 favor choosing 0

    p[t] = inv_logit(eta[t]);
    // Convert log-odds into a probability between 0 and 1

    m_now = m_now + alpha * (Other[t] - m_now);
    // Update memory using an exponential / Rescorla-Wagner style learning rule
    // Prediction error = observed opponent choice - current memory

    memory_after[t] = m_now;
    // Store the updated memory after observing the opponent's choice
  }
}

model {
  alpha ~ beta(2, 2);
  //alpha ~ beta(3, 3);
  // Prior for learning rate:
  // keeps alpha between 0 and 1 and mildly favors middle values over extremes

  beta ~ lognormal(0, 0.5);
  //beta ~ lognormal(0, 0.3);
  // Prior for sensitivity:
  // ensures beta is positive and favors moderate values

  bias ~ normal(0, 1);
  // Prior for side bias:
  // centered at no bias, while allowing moderate positive or negative bias

  m0 ~ beta(2, 2);
  // Prior for initial memory:
  // keeps m0 between 0 and 1 and favors starting beliefs near the middle

  for (t in 1:T) {
```

```
    Self[t] ~ bernoulli_logit(eta[t]);
    // Likelihood:
    // the observed choice on each trial is modeled as a Bernoulli draw
    // with log-odds given by eta[t]
  }
}

generated quantities {
  array[T] int y_rep;
  // Posterior predictive choices simulated from the fitted model

  vector[T] log_lik;
  // Log-likelihood of each observed trial, useful for model comparison or diagnostics

  for (t in 1:T) {
    y_rep[t] = bernoulli_logit_rng(eta[t]);
    // Generate one replicated choice for each trial from the fitted model

    log_lik[t] = bernoulli_logit_lpmf(Self[t] | eta[t]);
    // Store the pointwise log-likelihood of the observed choice
  }
}
```